

Keeping a Passed-Through USB Storage Device Alive

across Same-Host VM Live Migration with a Multi-Client vfiio-user Backend

Author Name
Department, University
Email: author@example.edu

Supervisor Name
Department, University
Email: supervisor@example.edu

Abstract—Passing a physical USB device through to a virtual machine (VM) is attractive for storage, dongles and instrumentation, but it conflicts with live migration: the host-side device session (open file descriptors, claimed interfaces, endpoint state and the device’s internal state) cannot be serialised, so any migration path that tears the session down forces the guest to re-enumerate. This paper studies the concrete case of Cloud Hypervisor together with `usbvfiio`, a Rust vfiio-user server that emulates an eXtensible Host Controller Interface (xHCI) controller. We first show, by measurement, that migrating a VM with a vfiio-user device *deadlocks*: the destination blocks for 199.9s inside the protocol version handshake because the vfiio-user server accepts exactly one connection, which the source still holds. We then present a design that removes the deadlock without modifying virtual machine monitor (VMM) source code, although the VMM must be rebuilt against the fixed protocol crate: because the vfiio-user handshake commands never touch the device backend, a wrapper that takes the backend lock *per command* rather than per connection lets a second client connect while the first client is still connected. We identify and fix two defects that the hand-over exposes—a departing client tearing down the surviving client’s interrupt line and DMA mappings, and completion interrupts lost inside the hand-over window—plus a reset-capability parsing bug in the protocol crate, and we describe a measurement harness that derives the verdict from the guest’s own log rather than from host-side observation. On a real 32 GB exFAT stick passed through to an Ubuntu 22.04 guest, a 128 MiB file copy that spans the migration completes in 10 out of 10 runs with a measured downtime of 4–18ms, byte-identical data and, at the USB level, no re-enumeration, no reset and no I/O error in the guest.

Index Terms—virtualisation, live migration, device passthrough, vfiio-user, USB, xHCI, Cloud Hypervisor, KVM

I. INTRODUCTION

Virtual machines are routinely equipped with physical devices through passthrough so that they can use hardware that is either impossible or unattractive to emulate. Live migration—moving a running VM between VMM instances with a downtime short enough to be unnoticeable—is the standard mechanism for host maintenance, load balancing and upgrades [1]. The two techniques, however, interact badly.

Memory and CPU state migrate well, and in-kernel VFIO devices can migrate when they implement the VFIO device

migration interface [2]. USB devices are the difficult case [3]. A guest talks to a USB device through a host controller; a passthrough implementation either emulates that controller and forwards individual transfers, or hands the whole controller to the guest. In the first case the host process that owns the physical device holds a session that is *not* part of the guest state: an open file descriptor, a claimed interface, endpoint configuration and, above all, the device’s own internal state. None of this can be serialised. Any migration that recreates the device on the destination therefore forces the guest to observe a disconnect and a re-enumeration, which at minimum interrupts I/O and can invalidate host-side state such as a mounted filesystem or an encrypted volume.

Cloud Hypervisor (CH) supports USB passthrough through the vfiio-user protocol via `--user-device socket=...` [4]. The server side can be `usbvfiio` [5], which emulates an xHCI controller and forwards transfers to the physical device with `nusb` [6]. CH documents live migration [7], but the vfiio-user path was never exercised under migration.

This paper starts from a measurement: migrating a CH VM that has a vfiio-user device attached does not fail cleanly—it *hangs*, and it hangs on the destination VMM (Section III). We then design and implement a solution that makes a same-host migration of such a VM work, and we measure it end to end with real hardware.

A. Contributions

- 1) We characterise, by measurement and source inspection, why a vfiio-user device deadlocks a CH live migration: the destination VMM blocks in the version handshake because the server accepts a single connection that the source still holds. We quantify it (199.9s of zero progress; the destination never gets a reply) and show that even a successful takeover would not transfer device state (Section III).
- 2) We present a *multi-client* vfiio-user backend design that removes the deadlock without modifying the VMM. The key observation is that the handshake commands never reach the device backend, so the backend lock

can be taken per command instead of per connection (Section IV).

- 3) We identify and fix two defects that only appear once a hand-over is possible—a departing client disabling the surviving client’s interrupt line and unmapping its DMA regions, and completion interrupts being delivered to the departing VMM’s interrupt descriptor inside the hand-over window (Sections IV-C and IV-D)—and a reset-capability parsing bug in the protocol crate that made every connection reset the device (Section V-B).
- 4) We build a reproducible end-to-end harness, including a guest image and a measurement method whose verdict is read from the *guest’s own log* rather than from host-side observation, and we report the design decisions that this method forced us to change (Section V).
- 5) We evaluate the result with a real USB stick: 10 out of 10 migrations of a VM that is copying a 128 MiB file at the time of the migration succeed with 4–18 ms downtime, byte-identical data and no re-enumeration or reset in the guest (Section VI).

B. Paper organisation

Section II reviews live migration, device passthrough and USB passthrough alternatives. Section III analyses the measured deadlock. Section IV presents the design, Section V the implementation. Section VI evaluates, Section VII discusses limitations, and Section VIII concludes.

II. BACKGROUND AND RELATED WORK

A. Live migration

Live migration of VMs was popularised by Clark *et al.* [1], who introduced pre-copy migration: pages are copied while the VM runs, and a final stop-and-copy phase transfers the remaining dirty pages while the VM is paused. KVM [8] made this a mainstream kernel mechanism, and modern VMMs such as Firecracker [9] and Cloud Hypervisor [10] implement variants of it. Cloud Hypervisor documents both TCP and UNIX-socket transports; for same-host migration it offers `memory_mode=memfds`, which passes the guest *memory backing file descriptors* (`memfd`, [11]) over the socket so that source and destination map the same memory instead of copying it [7]. This is the mode we use: it makes the memory identical in both VMMs, which is what allows a `vfio-user` server on the host to keep its DMA mappings.

B. Device passthrough: VFIO and vfio-user

VFIO is the Linux framework that exposes a device to user space with an IOMMU-backed DMA model [12]. In the kernel-VFIO model the VMM maps device BARs and registers event descriptors directly. `vfio-user` moves the same programming model to a user-space server reached over a UNIX socket [13]: a client (the VMM) queries regions, reads and writes device memory, maps guest memory for device DMA, registers interrupt file descriptors, and may reset the device. The protocol specification is maintained with QEMU [13] and reference implementations exist in `libvfio-user`

[14]. Cloud Hypervisor implements the client side [4]; the `vfio_user` crate [15], part of the `rust-vmm` collection [16], implements both sides and is used by CH and by `usbvfiod`.

Two lines of work are directly adjacent to ours. A QEMU patch series proposes protocol-level live migration for `vfio-user` devices by reusing the generic VFIO v2 migration infrastructure—migration data read/write and DMA logging for dirty-page tracking [17]. That is the right long-term answer, and it is *not* what this paper does: it transfers device state, which requires both ends to run a compatible server and does not preserve a host-side session that cannot be expressed as state. Our problem is the same-host case, where the server process can simply stay alive and only the VMM that talks to it changes; what we contribute there is the removal of the connection-level serialisation that otherwise makes the hand-over impossible, plus the hand-over correctness rules that a state-transfer design would also need. The second line is earlier work on migrating USB devices between hosts through a proxy mechanism [18], which likewise assumes a medium that carries the device rather than preserving a local session.

C. USB passthrough alternatives

Table I compares the options we considered. Passing through with a user-space server over `vfio-user` is the approach of `usbvfiod` and the subject of this paper: it keeps the guest driver unmodified and confines the device to an unprivileged process, at the cost of an emulated controller in the transfer path. USB/IP [19], [20] tunnels USB traffic over a network and is the natural answer for *cross-host* device sharing, but the device remains attached to the source host and depends on an external transport. QEMU also provides USB to a guest, either by emulating devices or by attaching a physical one with `usb-host` [21]; a passed-through USB host device cannot be interchanged during a migration [22], and USB redirection sends the traffic to a remote client with its own reconnection semantics [23] rather than preserving a host-side session.

D. Emulated xHCI and USB mass storage

`usbvfiod` presents an xHCI controller [24] to the guest. The Linux xHCI driver discovers the controller, performs a host controller reset, enumerates the attached devices and then drives transfers through command, event and transfer rings in guest memory. Mass-storage devices use the Bulk-Only Transport protocol [25], which the kernel’s `usb-storage` module implements. Our measurements use this path because it is the most common passthrough workload and because a bulk transfer makes the consequences of a lost completion immediately visible as an I/O error.

Figure 1 summarises the resulting architecture. The physical device is owned by a host process for the entire lifetime of the VM, including the migration; the VMM is only a client of that process, and the guest sees an ordinary controller.

III. PROBLEM ANALYSIS

A. Experimental setup

All measurements were taken on one bare-metal host running Debian 13 with Linux 6.17.4 (Proxmox kernel), 8 cores

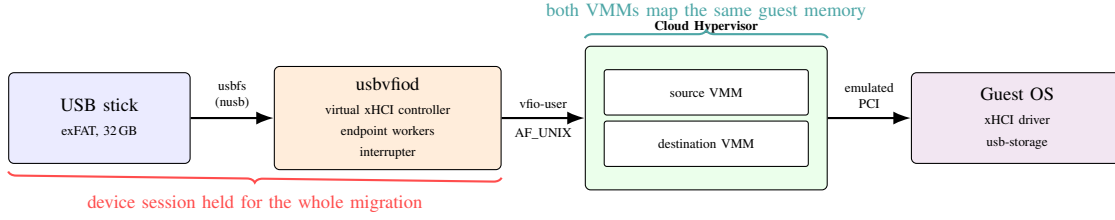


Fig. 1. System architecture. `usbvfio` owns the physical USB device and emulates an xHCI controller; the VMM reaches it over `vfiio-user`. During a same-host migration the server process is never restarted, so the device session and the controller state survive, and both VMMs map the same guest memory.

TABLE I
USB PASSTHROUGH APPROACHES.

Approach	Where device state lives	Consequence for migration
Host <code>usbfs</code> process with an emulated xHCI in the guest [5]	host process, not serialisable	session must survive; used here
USB/IP [19], [20]	source host, exported over a network	device stays on the source host; transport in the data path
QEMU <code>usb-host</code> (physical passthrough) [21], [22]	device bound inside the VMM process	a passed-through USB host device cannot be interchanged during migration [22]
USB redirection (<code>usbredir</code>) [23]	device stays on the host, traffic goes to a remote client	the client reconnects the transport; no host session is preserved
Kernel VFIO migration (non-USB) [2]	device-defined migration state	requires device support; not applicable to USB

and 62 GiB of RAM, which runs the Kernel-based Virtual Machine (KVM). The host was *not* idle: four unrelated KVM virtual machines (16, 8, 2 and 1 GiB of RAM) were running throughout, and no CPU pinning was used. We report this because it is an uncontrolled load confounder and, as Section VI-B notes, the most likely explanation for the spread in the measured downtime. Table II lists the software under test. The guest has 2 GiB of RAM and one vCPU, runs Ubuntu 22.04.4 (kernel 5.15.0-94-generic) and is booted directly by CH. The physical device is a 32 GB USB 2.0 High-Speed stick (Innoston 1f75:0903, exFAT data partition) attached at `/dev/bus/usb/001/007`. The server binary under test was an unoptimised cargo debug build, run with verbose logging (`-v`) and USB packet capture enabled; Section VI-G discusses the resulting measurement overhead.

B. Measured baseline: the migration deadlocks

Figure 2 illustrates what we measured. A source VM with `--user-device socket=...` is migrated with `send-migration using destination_url=unix:...` and `memory_mode=memfds`. The destination VMM starts from the migrated configuration and reaches the point where it restores the `vfiio-user` device:

TABLE II
SOFTWARE UNDER TEST.

Component	Version
Cloud Hypervisor	v53.0-520-gc24527002 (crate version 54.0.0) [10]
<code>usbvfio</code>	baseline 4d2c5af [5]
<code>vfiio-user</code> crate	0.1.5 [15]
<code>nusb</code> crate	0.2.7 [6]
Guest Device	Ubuntu 22.04.4, kernel 5.15.0-94-generic 32 GB exFAT USB stick, USB 2.0 High-Speed

```
device_manager.rs:4749 Restoring virtio-pci
_vfiio-user0 resources
```

and then makes no further progress. We waited 199.9 s; there was no additional log output, no error, and the destination VMM’s control API stopped answering. The source VM remained paused in the migrating state. Three observations pin the cause down: the server logged exactly *one* client version handshake (the source’s), a socket inspection showed `Recv-Q=1` on the listening socket, i.e. the destination’s connection was established but never accepted, and the destination had blocked inside

```
pci/src/vfiio-user.rs:4338 →
vfiio-user::Client::new()
→
negotiate_version() → read_exact()
```

waiting for a reply that never comes.

C. Root cause: one connection, one accept

The server side of `vfiio-user` 0.1.5 serves a connection by calling `listener.accept()` once and then looping over commands until the peer closes. A second client therefore *connects successfully* (the kernel completes the handshake into the listen backlog) but is never served, which is exactly the “hangs forever” symptom above. A cross-host migration would have the same problem, and additionally the destination would have no device: the guard for that exists only for kernel-VFIO devices (VFIO device does not support migration), not for `vfiio-user`.

D. Device state would not be transferred anyway

Even with the hand-over solved, the `vfiio-user` device carries almost no migration state in this version. Its `Pausable`, `Transportable` and `Migratable` implementations are empty, and its snapshot contains only the generic PCI state (configuration space, INTx/MSI/MSI-X configuration and the

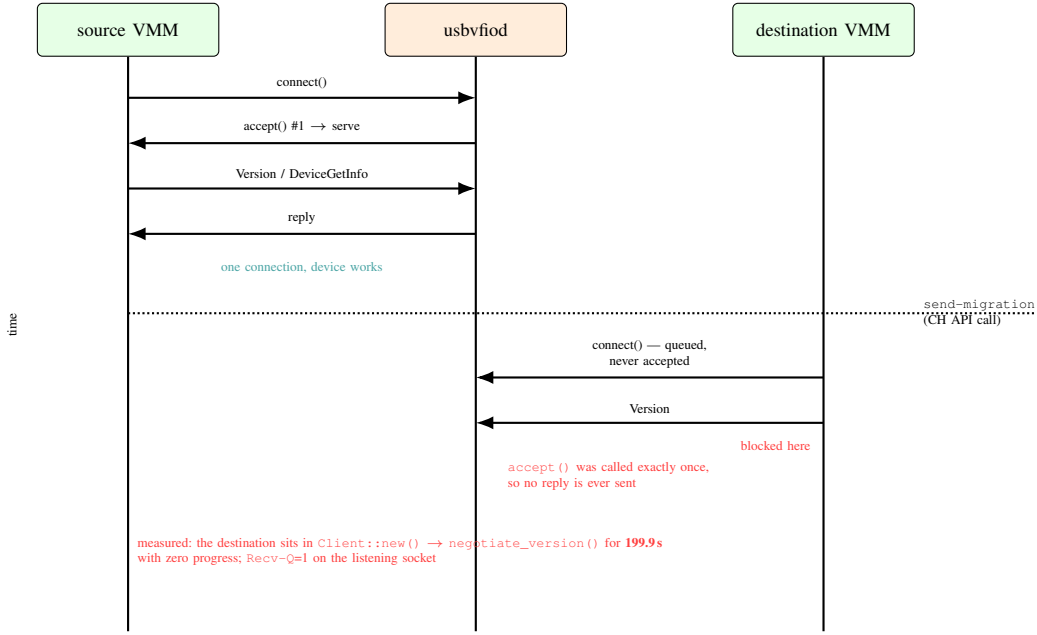


Fig. 2. Measured deadlock of the unmodified stack. The vfio-user server calls `accept()` exactly once, so the destination’s connection is queued and its version handshake is never answered; the migration never completes.

MSI-X table); device BAR state, opaque device state and DMA dirty tracking are absent. There is no device-state region in the protocol implementation and no parameter with which the destination could be pointed at a different vfio-user server. A protocol-level migration mechanism has been proposed for vfio-user [17], but the implementation used here carries no device state. This is why our design deliberately does *not* try to move device state: for a same-host migration the server process simply stays alive and keeps the session, and only the *client* that the server talks to changes.

IV. DESIGN

A. Goals and non-goals

The goal is that a same-host live migration of a VM with a passed-through USB storage device completes, and that the guest does not observe a device change. Concretely we require: (i) the migration completes; (ii) the guest sees no re-enumeration and no reset; (iii) data transferred across the migration is neither corrupted nor lost (duplication is not tested, as Section VI-G explains); (iv) the VMM needs no source change, being only rebuilt against the fixed protocol crate. Non-goals are cross-host migration, USB classes other than bulk storage, and isochronous transfers.

B. Multi-client backend with a per-command lock

The deadlock exists because the server serialises *connections* even though the commands it handles are mostly independent of the device. In the crate, only `RegionRead`, `RegionWrite`, `DmaMap`, `DmaUnmap`, `SetIrqs` and `DeviceReset` invoke the backend; `Version`, `DeviceGetInfo`, `DeviceGetRegionInfo` and `GetIrqInfo` do not. Consequently a second client

can complete its handshake while the first is still attached, provided that the backend is shared with a lock taken *per command*.

Figure 3 shows the resulting structure. A shared state object owns the backend and the interrupt-ownership bookkeeping, and hands out one handle per connection. Each handle implements the server backend trait by delegating to the shared backend under a mutex for the duration of a single call. The accept loop is replicated over a configurable number of threads, so the process can serve the source and the destination simultaneously. The default keeps the historical single-client behaviour unchanged, which matters for the systemd integration where the server is expected to exit when its only client goes away.

C. Ownership: a departing client must not tear the device down

Allowing two clients exposes a hazard that a single-client server never had. When the migration completes, the *source* VMM shuts its device down and sends two destructive commands: `SetIrqs` with an empty descriptor list (disable interrupts) and `DmaUnmap` for the regions it mapped. Both arrive *after* the destination has already registered its own interrupts and DMA mappings, and both would be applied to the shared backend. The result we measured is severe: the destination is left with a dummy interrupt line, no completion is ever signalled, and the guest concludes that the controller is dead and aborts the running I/O.

The wrapper therefore tracks which connection most recently registered interrupts and lets only that connection issue the destructive counterparts. Teardown from any other connection is logged and ignored. In every run we observed

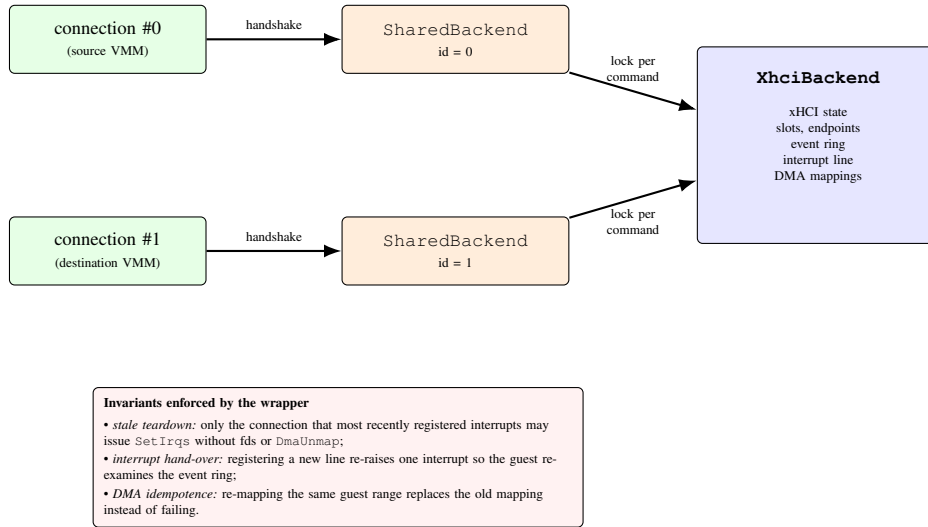


Fig. 3. Multi-client backend. Handshake commands never take the lock, so the destination VMM can connect while the source is still attached; the device state itself lives in the shared backend and therefore survives the hand-over.

the interrupt-disable half of this guard fire (the server logs the ignore); we did not observe a `DmaUnmap` reach the server on this path, so that half is implemented but not exercised by our workload. This is deliberately coarse: it is correct for the hand-over case, which is the case that needs it, and it cannot be triggered accidentally by the active client.

D. Interrupt hand-over correctness

A subtler, timing-dependent defect remains even after Section IV-C. The interrupter signals every event by writing to the currently installed interrupt line. During a hand-over there is a window in which a transfer may complete *before* the destination’s line is installed: the completion event is written into the guest’s event ring (which is in guest memory and therefore already correct), but the interrupt is delivered to the event descriptor of the VMM that is exiting. The event is then never noticed: the guest does not re-read its event ring without an interrupt, the copy stalls, and—because the USB stack eventually tries to recover—the guest starts issuing device resets that the emulated controller correctly rejects as a context state error.

Our fix is to re-raise one interrupt immediately after a new line is installed. A spurious MSI-X interrupt is harmless: the guest inspects the event ring, finds the pending completion (or nothing) and continues. This converts a lost interrupt into at most one redundant ring scan.

E. DMA mapping idempotence

Because source and destination map the *same* guest memory under `memory_mode=memfds`, the destination re-issues `DmaMap` for ranges that the source already mapped. A naive “insert only” bus rejects the duplicate as an overlap. We make the DMA bus replace a mapping that starts at the same address, while still rejecting a genuine overlap at a different address, and we make a failed insert leave the bus untouched. We also

implement the unmap path, which the VMM invokes when it destroys a device and which was previously unimplemented.

V. IMPLEMENTATION

A. *usbvfiod*

The change is confined to six files (about 380 added and 30 removed lines): a new `shared_backend` module implementing the wrapper and the ownership rules; the accept-loop and lifecycle logic in `main.rs`; a `--max-clients` option; the idempotent DMA bus; and the DMA, reset and interrupt-registration paths of the xHCI backend. The device emulation itself is untouched. Unit tests were extended from 109 to 112, and the project’s strict lint configuration (`clippy` with all, nursery and cargo groups denied) passes.

B. *Protocol crate*

One behavioural bug in `vfio_user` 0.1.5 had to be fixed for the reset semantics to be sane. The client parses the device’s reset capability by comparing the masked flag word with *inequality*, which inverts the result: a server that advertises “not resettable” is reported as resettable, so the VMM issues a device reset on every connection, including on the destination’s connection during a migration. The fix is a one-line equality comparison. Because the crate is consumed by the VMM, the fix is applied to the VMM’s dependency graph rather than to `usbvfiod`, which only uses the server side.

C. *Cloud Hypervisor integration*

No CH source file was modified, but the result does require a *custom CH build*: the integration artefact is a `[patch.crates-io]` entry that points CH at the fixed crate, and reproduction therefore needs that build. Because the crate lives in a monorepo whose manifest also references a sibling `vfio-bindings` crate that has diverged from the

published version, the patch deliberately uses a variant of the fix branch in which that crate resolves from crates.io; otherwise two copies of `vfio-bindings` coexist and their types no longer unify. This is a build-level detail, not a design change, and it is documented so that it can be dropped once the fix is released upstream.

D. Guest image and measurement harness

Measurement honesty turned out to require as much care as the device code, and we record the decisions because they changed our verdict twice.

Guest image. The Ubuntu live-server ISO cannot be booted directly: its `initrd` insists on finding a live medium and ignores `root=`, and the minimal `squashfs` ships an empty `/lib/modules`, so `usb-storage` cannot be loaded after the switch. We therefore unpack the ISO, graft the module tree out of the `initrd`, rebuild the `initrd` without the casper hooks and pack a bootable `ext4` image. The `exFAT` driver, which real sticks use, lives in `linux-modules-extra` and is merged in explicitly.

Console capture. CH opens a serial `file= target` with `File::create()`, so the destination VMM truncates the log when it starts; the pre-migration evidence disappears exactly when it becomes interesting. We capture the console through a FIFO kept open by a reader that never sees EOF.

Verdict source. The migration also re-creates the destination’s serial device, which resets the guest TTY. Progress written only to the console then looks like a hang although the guest is fine. We therefore make the guest log to a file and treat the serial console as a best-effort mirror, and we compute the verdict from the guest’s own log after stopping the VMs and mounting the disk image—with the journal replayed, because a read-only `noload` mount hides the most recent writes. A guest heartbeat service, finally, distinguishes “guest frozen” from “console broken”.

VI. EVALUATION

A. Methodology

The workload is a 128 MiB file copied from the passed-through stick to the guest’s own disk with `dd`, in a systemd unit that starts at boot. The harness waits until the guest reports that the copy has begun, waits a further four seconds so that the transfer is in flight, and then migrates the VM. A run is successful only if all four of the following hold: the copy window strictly contains the migration instant; the MD5 of the copied file equals the MD5 of the source file on the stick; the guest shows no enumeration after boot; and no reset or I/O error appears. The MD5 comparison is made inside the guest against the stick itself, so it also verifies that the device remained readable after the migration. The migration was requested with CH’s `downtime_ms=300` as the per-iteration stop-and-copy goal; the separate project acceptance target, used for the verdict, was ≤ 2000 ms. We ran the experiment 10 times after the fixes described in Section IV were in place. All ten runs were executed consecutively on the same host

with the same stick and the same 128 MiB file, whose content was generated once and never regenerated between runs.

B. Results

All 10 runs pass. Table III lists the per-run figures; Figure 4 shows the downtime distribution. Downtime is measured by CH (`downtime_ms`), not by the harness. Copy duration is the time the guest takes to copy the file, which is chosen to be long enough that the migration necessarily falls inside the copy window.

How much does 10 out of 10 establish? Not as much as the number suggests, and we prefer to say so. Ten consecutive successes give a one-sided 95 % lower confidence bound of 0.74 on the pass probability, i.e. the one-sided 95 % upper bound on the residual failure rate is 0.26; the two-sided 95 % Clopper–Pearson interval is [0.69, 1.00]. Against the pre-fix behaviour we observed—one failure in five runs—the comparison is not statistically significant (Fisher exact, $p \approx 0.33$). The honest reading is that the fixes removed a failure we could reproduce before them, and that the experiment bounds the residual failure rate only weakly. Section VI-G discusses what would be needed to do better; a deterministic injection test at the hand-over is the appropriate instrument, not more repetitions of a naturally occurring race.

Was the hand-over path actually exercised? Yes, in every run, and the harness records it. Each run shows two interrupt-line installations (the source registering at start-up, the destination taking over during the migration; between 2 and 2 per run) and one stale teardown that the server refused (between 1 and 1 per run, logged as `ignoring IRQ disable from stale vfio-user client`). Without that refusal the destination would have been left with a dummy interrupt line, which is the failure mode we measured before the guard existed. The counts come from the server log of each run, which is archived with the raw data.

The downtime figures are also less precise than their range suggests: the median is 15.5 ms but the order-statistic 95 % confidence interval for it spans essentially the whole observed range (4–18 ms). The distribution is bimodal and time-blocked—the three consecutive runs 7–9 gave 4, 4 and 6 ms while every other run gave 15–18 ms—which points to a time-varying environmental factor (the four co-resident VMs of Section III-A) rather than to independent noise. Copy duration varied between 13.2 s and 52.4 s; a no-migration control run is reported below, so we do not attribute that variation to the migration.

C. Timeline of a representative run

Figure 5 shows the guest’s copy progress for a demonstration run (not one of the ten acceptance runs) together with the migration instant. The copy curve is not perfectly smooth, but it continues across the migration and finishes with `rc = 0`; the guest never restarts the transfer. The final throughput in this run was 5.7 MB/s (5.4 MiB/s), consistent with the stick’s USB 2.0 read performance measured outside the migration.

TABLE III
ACCEPTANCE RUNS (POST-FIX). “SPANS” MEANS THE COPY WINDOW
CONTAINS THE MIGRATION INSTANT; “ENUMS” COUNTS GUEST
ENUMERATIONS AFTER THE MIGRATION UPTIME.

Run	Downtime (ms)	Copy (s)	Spans	MD5 / enums
1	17	20.6	yes	match / 0
2	18	30.2	yes	match / 0
3	15	33.9	yes	match / 0
4	17	33.5	yes	match / 0
5	15	35.3	yes	match / 0
6	16	32.4	yes	match / 0
7	4	28.4	yes	match / 0
8	4	13.2	yes	match / 0
9	6	15.0	yes	match / 0
10	17	52.4	yes	match / 0

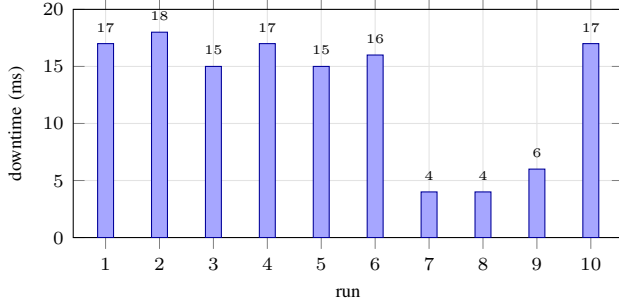


Fig. 4. Measured downtime of the ten acceptance runs. The goal of ≤ 2000 ms is far above the plotted range.

D. Data-path continuity under the hand-over

Figure 6 compares the USB packet rate across the hand-over before and after the interrupt fix, both taken from the vfiio-user server’s packet capture. Before the fix the rate drops to zero at the hand-over and remains near zero: the device is still powered and the server still holds it, but no further transfer is initiated because the guest is waiting for a completion it was never told about. After the fix the rate continues across the hand-over and only falls to zero when the copy has finished. This figure is the clearest evidence that the failures we saw were a lost-notification problem rather than a transport or device problem.

E. Defects found

Table IV summarises the four defects that had to be fixed. The first two are deterministic and reproduce on every run; the third is timing-dependent and appeared in roughly one run in five before it was fixed.

F. Discussion of the measurements

Two points about the method are worth stating explicitly, because both changed our conclusions during the work. First, host-side observation is not sufficient: the packet capture and the VMM logs looked healthy in a run whose guest had in fact stopped making progress. Second, a single passing run is not evidence for a timing-dependent fix: the code without the fix of Section IV-D passed four consecutive runs before the

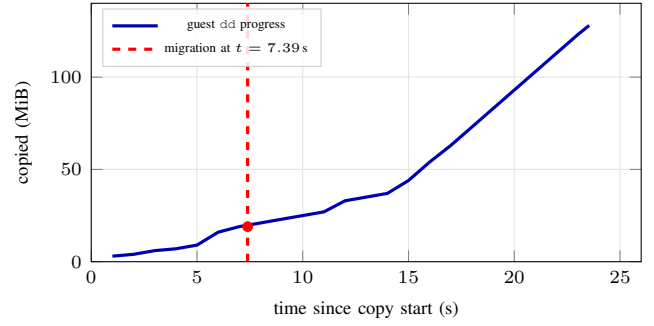


Fig. 5. Guest copy progress in a demonstration run (128 MiB), not one of the ten acceptance runs of Table III. The migration occurs at $t = 7.39$ s, inside the copy window; the copy completes at $t = 23.54$ s with status 0.

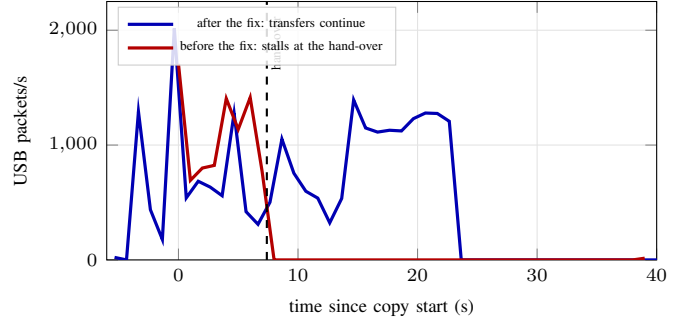


Fig. 6. USB packets per second across the hand-over, counted by usbmon at the vfiio-user server. Both traces are single runs, so this is an illustration of the mechanism rather than a controlled A/B comparison. Without the interrupt fix the transfers stop at the hand-over and do not recover within the observed window; with the fix they continue until the copy completes.

defect appeared on the fifth, and only repeated runs after the fix justify the claim of 10 out of 10.

G. Threats to validity

Construct. The integrity check is an MD5 of a read-only bulk copy that was read once, sequentially. It detects corrupted or lost data, but it cannot detect *duplicated* reads: a replayed read changes nothing in the destination file. Our claim is therefore *no data corruption and no lost I/O*, and we do not claim to have tested duplication. The enumeration and reset criteria are now derived from the guest’s own kernel log and are counted relative to the guest’s uptime at the migration instant (Section V), which removes an earlier threshold artefact in our harness, but they still rely on the kernel printing a message.

Internal. The host was loaded by four other VMs and no per-run load was recorded; run order was not randomised and the low-downtime runs happened to be consecutive. The server was a debug build with verbose logging and packet capture enabled. The migration instant used for the “spans” test is the instant the migration was *requested* on the host, converted through the heartbeat pairs rather than through a verified clock offset. The destination of the copy was a file on the host-backed guest disk, so host page-cache writeback is in the loop.

TABLE IV
DEFECTS FOUND AND FIXED.

Defect	Symptom	Fix
single-accept server	destination blocks 199.9s in the handshake	per-command lock, multi-client accept
stale teardown	guest declares controller dead after migration	interrupt-ownership rule
lost hand-over interrupt	copy stalls with no error (intermittent)	re-raise one interrupt on registration
inverted reset-capability parse (crate)	every connection issues a device reset	equality test instead of inequality

External. One device class (bulk storage), one USB 2.0 Bulk-Only Transport device on one controller, one guest kernel, one workload size, and same-host `memfds` migration only. A USB 3.0/UAS device with several in-flight commands would exercise the hand-over differently, and the slow $\sim 2\text{--}6\text{ MiB/s}$ transfer rate of the stick used here makes a completion less likely to land inside the hand-over window than it would be on faster hardware.

Statistical. $n = 10$, so the confidence intervals are wide, and the pre-fix rate is estimated from only $n = 5$; the statistical power is discussed in Section VI-B.

VII. DISCUSSION AND LIMITATIONS

a) What is transparent.: At the USB level the guest is undisturbed: it enumerates the device once, at boot, and never again; it observes no reset, no port status change and no I/O error; the file it copies is byte-identical to the source. This is the property that matters for storage passthrough and it is the one we set out to obtain.

b) What is not transparent.: One guest-visible disturbance remains and we do not want to hide it: after the migration the guest reprints its login banner. The destination re-creates the serial device, which resets the guest TTY. This is a property of the VMM’s serial device model, not of the USB path, and it does not affect the device. We report it because a claim of “zero perception” should be stated at the granularity at which it is true.

c) No explicit quiesce.: We did not implement a bounded drain of in-flight transfers before the hand-over. Transfers that are in flight when the migration happens are handled by the re-raise mechanism plus the guest’s own retry logic; the guest may therefore in principle have to retry a command, although in our runs the completion arrived through the re-raise and the copy was not restarted. A production implementation would add an explicit quiesce to bound this, and a guest-cooperative handshake if the downtime budget were tighter.

d) Scope.: The evaluation covers bulk storage on a USB 2.0 stick, one device per controller, Linux guests, and same-host migration. Other classes (HID, isochronous audio, serial) have different in-flight semantics; isochronous traffic in particular cannot be made lossless by retrying and is

not supported by the underlying `nusb` library. Cross-host migration is a different problem: it needs the device state to be transferred or an equivalent device to be provided on the destination, and it cannot preserve a host-side session.

e) Failure and rollback.: Ownership of the device is decided by whichever connection most recently registered interrupts, and it is not committed to a migration transaction. If a migration is cancelled after the destination has installed its line, the source is left running with a line that will not be serviced, and nothing re-registers it. Our own Phase-0 measurements showed that CH’s `timeout_strategy=cancel` did not in fact bring the source back to a usable state. A production design needs an explicit epoch that is committed at migration commit time, plus a revoke and re-register path, and it needs to be tested with a destination crash and a source-side cancel while I/O is in flight. We did not implement or test that.

f) Socket trust and bounds.: The `vfi`-user socket grants whoever can connect it full DMA authority over the guest’s memory, so it is as sensitive as the guest’s own memory. Our accept loop bounds how many connections are *served* (`--max-clients`), but a client beyond that limit waits in the listen backlog, there is no per-client timeout, and a client that registers interrupts becomes the owner and may then unmap DMA regions. A deployment must therefore treat the socket as trusted and protect it with filesystem permissions or a private namespace; the design does not defend against a hostile local client.

g) Process lifecycle.: With more than one client the server no longer exits when the last client disconnects, which is a deliberate change needed for the hand-over. Deployments that rely on exit-on-disconnect (for example socket-activated systemd units) must either keep the default single-client mode or manage the process explicitly.

VIII. CONCLUSION AND FUTURE WORK

We set out to make a USB storage device survive a live migration of the VM it is passed through to, and we found that the path did not merely fail—it deadlocked, because a `vfi`-user server built for one client cannot serve the destination while the source is still attached. The fix follows from a property of the protocol that is easy to miss: the handshake does not touch the device, so the device lock does not have to be held for the lifetime of a connection. On top of that, two further defects had to be fixed before the hand-over was correct rather than merely possible: ownership of destructive teardown, and the synchronisation of the interrupt line across the hand-over. With those in place, a 128 MiB copy that spans the migration completes in 10 out of 10 runs with 4–18 ms of downtime and byte-identical data.

Future work falls into three groups. First, an explicit quiesce and a guest-cooperative protocol would make in-flight behaviour deterministic instead of relying on retry. Second, a protocol-level device-state region would allow the device to be reconstructed on another host, which is the prerequisite for cross-host migration; our measurements show that the current

protocol carries none of the required state. Third, the same technique should be evaluated for other device classes and for multiple devices on one controller, where the ownership rules we introduced would need to become per-device.

APPENDIX A REPRODUCTION

The full harness, the development log and the raw measurement data are in the project repository at <https://github.com/yeungtuzy/usbvfiod> (the guest harness under `guest/`, the paper and its figures under `paper/`, and the development log under `docs/DEVLOG_cn.md`). In outline:

- 1) build the guest image: `guest/build-guest.sh`;
- 2) place a 128 MiB `testfile.bin` on the stick and record its MD5 in `guest/testfile.md5`;
- 3) run `guest/usb-migration-demo.sh` (add `PAUSE=1` for a step-by-step narrated run);
- 4) read the verdict, which is derived from the guest's own log as described in Section V.

REFERENCES

- [1] C. Clark, K. Fraser, S. Hand, J. G. Hansen, E. Jul, C. Limpach, I. Pratt, and A. Warfield, "Live migration of virtual machines," in *Proc. 2nd USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2005, accessed: 2026-09-20. [Online]. Available: <https://www.usenix.org/conference/nsdi-05/live-migration-virtual-machines>
- [2] Linux kernel, "VFIO device migration interface (include/uapi/linux/vfio.h)," kernel user-space API header; the migration commands and feature bits are defined there. Accessed: 2026-09-20. [Online]. Available: <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/include/uapi/linux/vfio.h>
- [3] The Linux Kernel documentation, "USB support," accessed: 2026-09-20. [Online]. Available: <https://docs.kernel.org/usb/index.html>
- [4] Cloud Hypervisor project, "VFIO-user HOWTO (docs/vfio-user.md)," accessed: 2026-09-20. [Online]. Available: <https://github.com/cloud-hypervisor/cloud-hypervisor/blob/main/docs/vfio-user.md>
- [5] Cyberus Technology GmbH, "usbvfiod: A vfio-user server for USB pass-through," accessed: 2026-09-20. [Online]. Available: <https://github.com/cyberus-technology/usbvfiod/tree/4d2c5af>
- [6] K. Mehall, "nusb: A new pure-rust library for cross-platform low-level USB," accessed: 2026-09-20. [Online]. Available: <https://github.com/kevinmehall/nusb>
- [7] Cloud Hypervisor project, "Live migration (docs/live_migration.md)," accessed: 2026-09-20. [Online]. Available: https://github.com/cloud-hypervisor/cloud-hypervisor/blob/main/docs/live_migration.md
- [8] A. Kivity, Y. Kamay, D. Laor, U. Lublin, and A. Liguori, "kvm: the linux virtual machine monitor," in *Proc. Linux Symposium*, vol. 1, 2007, pp. 225–230, accessed: 2026-09-20. [Online]. Available: <https://www.kernel.org/doc/ols/2007/ols2007v1-pages-225-230.pdf>
- [9] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa, "Firecracker: Lightweight virtualization for serverless applications," in *Proc. 17th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2020, accessed: 2026-09-20. [Online]. Available: <https://www.usenix.org/conference/nsdi20/presentation/agache>
- [10] Cloud Hypervisor project, "Cloud hypervisor: A virtual machine monitor for modern cloud workloads," accessed: 2026-09-20. [Online]. Available: <https://github.com/cloud-hypervisor/cloud-hypervisor>
- [11] Linux man-pages project, "memfd_create(2)," accessed: 2026-09-20. [Online]. Available: https://man7.org/linux/man-pages/man2/memfd_create.2.html
- [12] The Linux Kernel documentation, "VFIO — virtual function I/O," accessed: 2026-09-20. [Online]. Available: <https://docs.kernel.org/driver-api/vfio.html>
- [13] QEMU project, "vfio-user protocol specification," accessed: 2026-09-20. [Online]. Available: <https://www.qemu.org/docs/master/interop/vfio-user.html>
- [14] Nutanix, "libvfio-user: Framework for emulating devices in userspace," accessed: 2026-09-20. [Online]. Available: <https://github.com/nutanix/libvfio-user>
- [15] The vfio_user authors, "vfio_user 0.1.5 — rust crate documentation," accessed: 2026-09-20. [Online]. Available: https://docs.rs/vfio_user/0.1.5
- [16] rust-vmm project, "rust-vmm/vfio," accessed: 2026-09-20. [Online]. Available: <https://github.com/rust-vmm/vfio>
- [17] H. Komatsu, "[PATCH v2 0/8] vfio-user: Live migration support," QEMU development mailing list, September 2026. Accessed: 2026-09-20. [Online]. Available: <https://lists.gnu.org/archive/html/qemu-devel/2026-09/msg03538.html>
- [18] H. Watanabe, H. Masaoka, T. Ohgashi, T. Kondo, K. Nishimura, and R. Aibara, "Supporting USB devices for the global migration," in *Proc. 10th Annu. Int. Symp. Applications and the Internet (SAINT)*. IEEE Computer Society, 2010, pp. 153–156.
- [19] T. Hirofuchi, E. Kawai, K. Fujikawa, and H. Sunahara, "USB/IP: A transparent device sharing technology over IP network," *IPSIJ Digital Courier*, vol. 1, 2005, accessed: 2026-09-20. [Online]. Available: https://www.jstage.jst.go.jp/article/ipsjdc/1/0/1_0_394/_article
- [20] The Linux Kernel documentation, "USB/IP protocol," accessed: 2026-09-20. [Online]. Available: https://docs.kernel.org/usb/usbip_protocol.html
- [21] QEMU project, "USB emulation," accessed: 2026-09-20. [Online]. Available: <https://www.qemu.org/docs/master/system/devices/usb.html>
- [22] libvirt project, "Domain XML format — host device assignment," documents that a passed-through USB host device cannot be interchanged during migration. Accessed: 2026-09-20. [Online]. Available: <https://libvirt.org/formatdomain.html>
- [23] SPICE project, "usbredir: USB device redirection protocol," accessed: 2026-09-20. [Online]. Available: <https://www.spice-space.org/usbredir.html>
- [24] Intel Corporation, "extensible host controller interface for universal serial bus (xHCI) specification, revision 1.2," accessed: 2026-09-20. [Online]. Available: <https://www.intel.com/content/dam/www/public/us/en/documents/technical-specifications/extensible-host-controller-interface-usb-xhci.pdf>
- [25] USB Implementers Forum, "Universal serial bus mass storage class bulk-only transport, revision 1.0," 1999, accessed: 2026-09-20. [Online]. Available: https://www.usb.org/sites/default/files/usbmassbulk_10.pdf